

Hyperspherical Fitness and Lineage Cohorts: Recovering 75 of 133 SRBench Ground-Truth Laws on One Seed at Six Minutes a Fit

HFF-SR

23 September 2026

Abstract

We report 75 of 133 ground-truth laws recovered exactly on the SRBench ground-truth track at noise 0, using one development seed and 360 seconds per problem. The whole run took 6 h 04 m. AIFeynman, the best published entrant on this track, recovers 54.1% (72 of 133) using ten seeds at up to eight hours a problem; the next best, GP-GOMEA, recovers 27.1%. Our compute was 0.57% of the benchmark’s allowance.

The change that produced the result is a selection restriction we call virtual ALPS. Lineage instrumentation showed total diversity collapse: across four long fits the pump drew 294,000 fresh individuals and not one of their descendants survived. The cause was arithmetic, not competition — a 105-draw tournament on the measured configuration had a 6.7×10^{-11} chance of containing no converged elder, so young material was never sampled without one. We label each row with the pump beat its line arrived on, and tournaments prefer candidates of the same label. Strogatz recovery went from 3 of 14 to 7 of 14.

We also report what did not work: differentiated per-island rule sets lost, the general mutation beam is exhausted at a 0.05% hit rate, and combination masking generated 30,180 candidates a beat and converted nothing. Two limitations are stated plainly. This is one development seed, not the official ten. And the p-value half of our stopping criterion is inert at this operating point; the R^2 condition does all the work.

1 Introduction

SRBench’s ground-truth track asks whether a symbolic regression method recovers the equation, not whether it fits the data. There are 133 problems: 119 from the Feynman lectures and 14 from Strogatz’s nonlinear dynamics text. A problem counts as solved when SRBench’s own scorer, `assess_symbolic_model`, judges the submitted expression symbolically equivalent to the stored true model. A model with test R^2 of 0.999999 and the wrong shape scores zero.

This paper describes an engine built around four pieces: gene expression programming with Karva K-expressions evaluated on the GPU; hyperspherical fitness (HFF), which treats the objective vector as a point on a sphere and ranks by its angle from the origin where every error vanishes; an island model with a pump that refills an intake island from fresh random draws; and `fuller`, an egglog-based rewriter that replaced sympy in the engine’s inner loop. The result we report is 75 of 133.

The mechanism that got us there is one thing, not two. A search that converges onto a single line cannot be improved by handing it a better expression: the better expression is selected out before it can develop, which is the same starvation Section 4 measures for fresh random individuals. Cohort-restricted tournaments make space in the population for material that is not yet competitive. Rewriting is what makes that space worth having, because it is how a *proposal* gets made — a gene decodes to an expression, the expression is rewritten, and the rewritten form converts back into a gene. Neither half is the contribution on its own. A

system that makes space for edits, edits that are proposed rather than imposed, and selection that decides which of them survives: that is the claim. Section 4.5 states it in full, and states plainly which part of it we have measured and which part we have not.

Section 3 describes the engine. Section 4 is the main contribution: the diversity measurement, its arithmetic, the fix, and in Section 4.5 the interaction with rewriting and the experiment that would isolate it. Section 5 gives the run. Section 6 is the failure-class analysis, which reframed what we work on next. Section 7 covers data-guided rewrites, including one that recovered a law never solved in any prior run, and one we declined to loosen. Section 8 reports three negative results. Section 9 states the rules the run was held to, and Section 10 the limitations.

2 Related Work

3 The system

3.1 Genes on the device

An individual is a gene expression programming chromosome: one to three genes, each a Karva K-expression with a head of 34 symbols and a tail sized so that every head arrangement decodes to a valid tree. Genes combine through a linker — average, product or sum — and the linked result passes through an outer wrapper, one of identity, $\log|x|$ or $\sqrt{|x|}$, so the model as scored is $a \cdot W(L(g_0, g_1, g_2)) + b$ with a and b fitted by ordinary least squares in the same pass that evaluates the tree. The chromosome carries its best wrapper as it carries its best linker, and the wrapper travels with the row through selection and through every edit. The whole population is a flat buffer of `u32` tokens and `f32` constants; selection, mutation and crossover are four compute kernels, one dispatch each per generation.

Two decisions here matter later. The first is the dynamic gene-subset choice: a chromosome is scored under every non-empty subset of its genes, each subset under each linker, so 15 gene-linker combinations for three genes — 45 candidates once each is tried under each wrapper — and it keeps the best. A law that is one nested structure no longer needs the other two genes to evolve into exact do-nothing values. On Feynman I.26.2 the engine ran 328 generations without the subset choice and did not solve it; with it, the fit early-stopped at generation 14 using a single gene. It costs 2.7 times as much per generation.

The second is that the engine runs in `f32` on the device. Compensated summation was measured dead on this hardware — the Metal compiler removed the Neumaier correction and the device matched neither the compensated nor the plain CPU twin. After that was fixed, along with a 1-ulp correction to the device’s division and square root, the `f32` floor on a true law fell from 1.2×10^{-10} to 1.1×10^{-14} . The stopping bar sits at 10^{-10} , so this was not a cosmetic fix.

3.2 Hyperspherical fitness

The engine computes up to nine error objectives: mean squared error, $1 - R^2$ and mean absolute error, each on three blocks of rows (train, validation, and a third block that may be synthetic-noisy or edge rows). Each is capped at the constant model’s error on that block — a model worse than predicting the mean is useless, and how much worse carries no information — and scaled into $[0, 1]$ against frozen ranges.

The run reported here used six. The third block is dropped entirely when no synthetic or edge rows are generated, and the run generated none, so the objective set was mean squared error, $1 - R^2$ and mean absolute error on train and validation. The two non-error objectives below — the tower and a redundancy column — were both off. We state this from the run’s recorded settings and the engine’s defaults: the run emitted no card recording the derived objective count, and the card format that does postdates it.

The third block is where the two halves of the stop bar disagree, and we record it as an open design question rather than a resolved one. When the block holds synthetic noisy rows it exists to rank individuals in tournaments and is not meant to decide that a fit is exact — the $1 - R^2$ condition of Section 3.5 honours that and skips it, but the angle does not, so the block’s error enters p like any other objective. One of the two is wrong. It did not bite in this run, which had no third block at all, but it bears on the p-value discussion of Section 10.

HFF then treats the scaled vector as a point on the unit sphere and returns its angle from the pole where all errors are zero:

$$\theta = \arccos\left(1 - \min\left(\frac{1}{m} \sum_i x_i^2, 1\right)\right),$$

with m the number of objectives. Lower is fitter. There is no weighting, no scalarisation of the objectives into one number by hand, and no parsimony term in the fitness. Size is a reporting decision, taken after the search ends.

The pole matters and was chosen by measurement. HFF also admits a *balanced* pole — the angle from $(1/\sqrt{m}, \dots, 1/\sqrt{m})$, which rewards an individual for being even across its objectives rather than for being close to zero on all of them. Ranking tournaments from that pole scored 26 laws against 53 for the zero pole on the same seed with everything else equal, and the zero pole recovered 30 laws the balanced pole missed. That is a larger swing than any other single decision in this paper. The balanced pole remains available for selection and is off.

Objectives can be entered on a log scale, $1 + \log_{10}(x)/12$, and the argument for it is the accuracy we need. Squared as it stands, small errors weigh nothing: an error of 5×10^{-3} contributes 3×10^{-5} and one of 2×10^{-4} contributes 4×10^{-8} . A tournament cannot tell a plausible fake at 10^{-3} from an exact law at 10^{-12} . On the log scale those two sit at 0.75 and 0, and every factor of ten in the error counts the same. The log scale is off by default and was off in this run; the argument above is why it exists, not a measured contribution to 75.

The angle converts to a p-value: the probability that a point drawn uniformly on the m -objective hypersphere lies within θ of the pole, the regularised incomplete beta $I_{\sin^2 \theta}(\frac{m-1}{2}, \frac{1}{2})$. We report $\log_{10} p$ throughout, computed in log space so it stays finite deep in the left tail where p itself underflows.

One available objective is not an error. The *tower* objective is the transcendental nesting depth of the decoded gene — the most transcendental functions met on any path from root to leaf. Addition, subtraction, multiplication, division, negation, reciprocal and whole powers do not count, so a long flat law scores 0. It is off by default, and off in the run of Section 5; it was on in the lineage measurement of Section 4.1. We describe it because it is a component of the engine, not because it contributed to 75. This is not parsimony. It was calibrated on 1,230 labelled fits: every one of SRBench’s 133 true laws has a depth of at most 2, and three quarters of the accurate-but-wrong models are 3 or more. The penalty has a free zone up to depth 2 because charging from depth 0 makes the search settle for flat polynomials — it rebuilt $mc^2 + mv^2/2$ in place of $mc^2/\sqrt{1 - v^2/c^2}$ and never tried a root.

3.3 The pump and the islands

The population is organised in pairs. Each pair has an intake island and a champion island. On the pump’s beat, the intake’s best two individuals are copied over the champion island’s worst two; the intake then keeps its best fifth — one of each distinct row — and the rest is refilled with fresh random individuals. A cross step, off by default, lets each intake island take in the best of the other pairs’ champion islands.

The design intent is plain: the champion island accumulates, the intake island churns, and the pump is where diversity enters. Section 4 is about the measurement that showed it was not entering.

3.4 fuller: rewriting that converts back into the genotype

fuller is a Rust crate built on egglog 2.0 that replaced sympy in the engine. It holds a real-domain expression datatype with the engine’s protected operators as distinct functions from the raw ones, a real-domain evaluator whose semantics match the device exactly, and several rulesets: algebraic identities, powers, distribution, rational arithmetic and trigonometry.

The important structural point is that **fuller** converts in both directions. A chromosome decodes to an expression, the expression is rewritten, and the rewritten form converts *back into a Karva gene*. That return leg is what makes a rewrite available to the search rather than only to the report, and it is the capability the argument of Section 4.5 rests on.

Two rewriters use it, and only one of them ran inside the search that produced 75. *Constant snapping* is the one that did: a folded literal near π , e , $\sqrt{2}$ or a physical constant is matched against a resident lattice on the GPU, judged on whether the whole model’s R^2 survives, and written back into the gene in place. *Saturate-and-extract* — decode, run the e-graph to a bounded fixpoint, enumerate variants, score them on the data and keep the smallest within tolerance — is the algebraic half. It is implemented and exposed to callers over the crate’s Python interface, and it has no call site in the engine’s generation loop; it was a stated design for the search, not a component of this run. Section 4.5 is exact about which was which and why the distinction matters to the claim.

Two things about this substrate cost us time. Rule families are non-confluent: distribution co-saturated with trigonometry or with rational arithmetic explodes the e-graph and pegs a CPU core. The saturate-and-extract path therefore uses only a bounded algebra-plus-powers subset, and distribution is kept out of it entirely — it is a normal-form canonicaliser, and enumerating variants over its equivalence class does not terminate in useful time. And egglog does not constant-fold floating-point literals unless a rule says to, which is a correctness feature and a repeated surprise.

Against frozen sympy corpora, generated offline so that sympy is never in the scoring loop, **fuller** reaches about 33.6% parity overall: 84.9% on **powsimp**, 47% on **trigsimp**, around 15% on **radsimp** and **ratsimp**, and 13% on **simplify**. The low radical and rational figures are partly a measurement artifact — sympy rationalises to larger canonical forms, not the smaller expression a symbolic regressor wants — but they are not solved.

3.5 The stopping bar

A fit stops when *both* train and validation $1 - R^2 \leq 10^{-10}$ — and the third block’s too, when a third block exists and it is edge rows rather than synthetic ones — and $\log_{10} p \leq -19$. The run of Section 5 had no third block, so its bar was train, validation and p .

The p-value half was added after a measurement on development seed 7013: every real law that met the R^2 condition sat at -19.4 or below, most between -21 and -22 , and the one fake that slipped under the R^2 bar (Strogatz **bacres2**, validation $1 - R^2 = 1.0 \times 10^{-11}$) sat at -17.55 .

The train half was added later and for the same failure from the other side. A validation split can be lucky: over the cascade’s 117 early stops, all 113 real wins had train $1 - R^2 \leq 10^{-10}$ with a median of 9.8×10^{-15} , and the single exception was **bacres2** again — stopping with validation ten times better than the rows the model was actually fitted on, which is a description of noise rather than of a fit. Requiring both sides cost no true win on record. It was in place before the run reported here. Note what this means for the p-value’s job: the case that motivated the p condition is now also excluded on the error side, and Section 10 returns to whether p is still earning its place.

The bar is ours. It never reads SRBench’s verdict. That distinction does real work in Section 9.

4 Virtual ALPS: couples from the same century

4.1 The measurement

We instrumented every individual with an identity, an age (generations since its genotype entered the population, by Hornby’s ALPS rule: an offspring is its parent’s age plus one, a fresh random individual is 0) and a lineage back to the founder its line began at. The log is written only when a path is given; with it unset the engine is bit-identical to before and costs nothing. Nothing about selection changed for this measurement, and no age layers were added.

Six fits on seed 7013, population 1,500 intake plus 1,500 champion, head 34, 30 seconds each, with the tower objective on, no validation block, a constant range of ± 5 and the pump on every fifth generation, on a GPU shared with other running fits so the timings are under contention. Table 1 is the result.

law	generations	winner age	founder origin	founders, all rows
I_12_5 (solved, gen 1)	1	1	init	60
I_14_3 (solved, gen 1)	1	1	init	40
II_38_14 (solved)	204	204	init	1
I_39_11 (not solved)	335	335	init	1201
I_48_2 (not solved)	360	360	init	1201
III_4_32 (not solved)	332	332	init	1

Table 1: Six instrumented fits. Every winner’s founder was drawn in the initial population. *Founders, all rows* counts the distinct lines present across all 3,000 rows at the end of the fit.

Two readings, both stark.

Every winner’s founder was drawn in the initial population. Not one fit was won by a line the pump introduced later. That is consistent with an earlier finding that 54% of solves are already present in the initial population and 69% by generation 5.

The population converges onto one line and the pump never breaks it. Across the four long fits the pump drew 294,000 fresh individuals — 40 to 72 refills of roughly 1,200 rows each. Of the 73,990 rows later kept as the intake’s best fifth or promoted into a champion island, the founder origin was *init* for every single one. Zero came from a pump refill. A fresh draw’s descendants never once reached the top fifth of the island they were drawn into.

The founder count makes the timescale exact. *Founders, all rows* reads 1201 on precisely the two fits that ended *on* a pump beat ($335 = 67 \times 5$, $360 = 72 \times 5$): that is the 1,200 rows just drawn, plus one line for the other 1,800. On the two fits that ended off the beat it reads 1. Every one of 1,200 fresh lines is extinct across all 3,000 rows within two generations of arriving.

4.2 The arithmetic

This is not a competition result. It is a sampling result, and it is arithmetic.

A refilled intake holds 300 keepers — its best fifth, by construction fitter than anything just drawn — among 1,500 rows. The tournament size is 7% of 1,500, which is 105. The probability that a single 105-draw tournament contains no keeper at all is

$$\left(\frac{1200}{1500}\right)^{105} = 6.7 \times 10^{-11},$$

and selection is two-stage, so the real figure is tighter still. Young material is not outcompeted. It is never *sampled* without a converged elder in the same draw. No amount of fresh genetic material fixes this, because the fresh material was never the bottleneck.

This is why we are willing to carry the measurement across configurations. The six instrumented fits did not use the objective set or the constant range of the run in Section 5 — no validation block, the tower on, a narrower constant range that leaves constant snapping with little to bite on. The arithmetic above is the reason that does not matter here: it is a statement about tournament size against keeper fraction, and it contains no term for the objectives, for what is being fitted, or for how the winner is scored. Any configuration that refills a fifth of an island and draws 105-candidate tournaments from it starves young material at the same rate.

4.3 The change

Hornby’s ALPS solves this with physical age layers: the population is split into bands by age, an individual too old for its band is moved up or discarded, and the top layer is unbounded. We kept the selection restriction and dropped the physical layers.

Each row carries a cohort label: the pump beat its line arrived on, inherited by every descendant. It is a *lineage* cohort, not an age — a fresh mutation of an elder is still an elder, which is exactly what keeps young material from meeting one in a tournament. A tournament run by a row of cohort *c* prefers a candidate of cohort *c* over any other, however fit the other is.

The device implementation is branchless. Rather than reject and redraw, which would put a loop on the GPU, the cohort is the first sort key and fitness the second. This is the comparator as it stood for the run reported here, at commit 05ef7e9; it has since gained a per-island flag that can lift the restriction, and the run applied it on both islands of the pair:

```
fn better_mate(me: u32, c: u32, w: u32) -> bool {
    let mc = same_cohort(me, c);
    let mw = same_cohort(me, w);
    if (mc != mw) { return mc; }
    return key(c) < key(w);
}
```

A mismatched candidate simply loses. When a tournament contains no match at all, the best of the draw still wins, so a cohort that has died out costs nothing and needs no special case. The whole mechanism is one u32 per row and no extra dispatch.

`cohort_merge` is the threshold past which cohorts stop being separate: cohorts at or beyond it are clamped to one band, so the elders co-mingle freely and only the young are kept apart. That is Hornby’s unbounded top layer, expressed as a clamp. In the run reported here it was a clamp on the label itself (05ef7e9); the engine now clamps on age, `generation - label`, which is the intended reading and was a bug fix — banding on the label alone holds the oldest lines apart permanently and merges the youngest, the opposite of the design.

The merge was effectively never reached in the run either way. It was set to 10,000 against fits of a few thousand generations — at the rates of Section 5, a Feynman fit reaches about 4,300 generations in its 360 seconds and only the longest Strogatz fits cross the threshold at all. So the 75 was produced with the cohorts permanently separate and the unbounded top layer inactive. The elder merge is a design we describe and have not tested; whether the result depends on its absence is an open question, and a cheaper experiment than the one Section 4.5 asks for.

Setting `cohort_merge` to 0 turns cohorts off entirely, and the CPU/GPU bit-for-bit parity tests then prove the engine is what it always was. It must also exceed the pump period to mean anything: a merge that fires inside a single beat is the mechanism switched off wearing labels, which the engine asserts against.

4.4 What it measured

On one law — Feynman III.8.54, the hardest we had — the change is unambiguous. Measured over a single 500-generation fit at population 10,000 intake plus 5,000 champion, same seed both arms: surviving lines went from 1 to 483, the fraction of survivors whose founder came from the pump went from 0% to 100%, and $1 - R^2$ went from 6.70×10^{-1} to 1.12×10^{-1} . That is one law and one fit, not a sweep.

Strogatz **barmag2** is the clean proof. It had been stuck at $1 - R^2 = 6.9 \times 10^{-3}$ in a converged monoculture. With cohorts on it early-stops at generation 614 with $1 - R^2 = 6.66 \times 10^{-14}$.

Across the benchmark, Strogatz recovery went from 3 of 14 to 7 of 14. That is the clearest attribution in the run, because the cohorts were built for exactly this failure and Strogatz is where it bit hardest.

4.5 Space for editing, and what we have not measured

Cohorts and rewriting are presented above as two pieces of engineering. We think they are one mechanism, and the case for it is short.

The unusual capability is the round trip of Section 3.4: a gene decodes to an expression, the expression is rewritten into an equivalent form, and that form converts *back into a gene* the search can breed from. Most symbolic regression systems convert one way only — a genotype becomes an expression, the expression becomes a number, and the number is where it ends. Converting back is what makes a rewrite available to the search at all, rather than to the report.

It is worth being exact about which rewriter did that in the run reported here. The operator that writes into a live gene is the constant snap of Section 3.4: a folded constant is matched against the resident lattice, the substitution is kept only if the whole model's R^2 survives it, and the accepted form is written back into the gene in place. That is a repair of an individual rather than a proposal beside it — the row keeps its identity, its age, its line and its cohort, and the guard, not a tournament, is what accepted the form. The algebraic half of **fuller**, the saturate-and-extract round trip, is a capability the crate provides and a stated design for the engine; it is not a component of the run that scored 75.

The design the argument calls for is the stronger one: an edited form entering the population *beside* the original rather than over it, unevaluated, with the tournament as the only thing that judges it. Then nothing is imposed. The engine would have no way to declare a rewritten form better and would not try to; it would enter the pool as a candidate and be held to the same fitness as every other row. That is guidance through equivalence rather than steering by hand.

Since the run we have built one operator in that shape, and report it here because its result so far is not a good one. A constant-folding edit — a subtree that flattens its input to a constant is collapsed to that constant — lands the cleaned gene in its island's worst row, unevaluated, leaving the original untouched for the tournament to choose between them. Across three seeds at 400 generations it is one win and two losses, and we claim nothing from it: 400 generations is a poor test of an operator aimed at a constant blob carried for sixty thousand, so what the A/B establishes is that the mechanism runs, not that it pays. It also exposes a tension the argument above does not resolve: the proposal inherits its parent's cohort rather than a fresh one, so it fights in its parent's band and does *not* get the protected young band that this section argues a proposal needs. Which of those two a proposal should wear is the open question, and it is not the same question as the missing row below.

That is exactly the material Section 4 shows a converged population destroys. A proposal is, at the moment it is made, a row whose structure is untried. Dropped into a monoculture it would be drawn into a 105-candidate tournament against a converged elder with probability $1 - 6.7 \times 10^{-11}$, and it would be gone, however good the form it carries. Cohort restriction is what would give it a protected band to develop in before it has to answer to an incumbent. The two halves need each other in both directions: an edit without protection does not survive to be

judged on its merits, and protection without edits is age-layered selection over random variation, which is worth having only when there is something worth protecting.

We have not isolated this. The 75-of-133 run had both cohorts and snap on, and so did the cascade it is compared against. What the record supports is an attribution of the cohort change *given* editing on: the same-seed arm on Feynman III.8.54 in Section 4.4, the cascade at 66 against this run’s 75, and Strogatz 3 of 14 to 7 of 14. What the record does not contain is a single fit with editing switched off, in either cohort condition. The two-by-two has one row, not one cell, and the missing row is the one that would show whether the cohort gain depends on there being proposals to protect or would have arrived just as well over random variation alone. The negative result in Section 8.2 does not fill the gap: it measures a generated mutant against its own original at the moment it is made, which is a different question from whether a protected band lets a proposal develop into a solve. The experiment is cheap and we have not run it.

5 The run

One development seed (7014), 360 seconds per fit, 133 problems, 6 h 04 m total. The settings: 800 intake plus 400 champion, pump every 100 generations, `cohort_merge` = 10,000, gene subsets on, generation cap 50,000 so that time binds everywhere. The generation cap matters — Strogatz problems run at about 33 generations per second against Feynman’s 12, and a Feynman-sized generation cap was cutting Strogatz fits off with a third of their time budget unspent.

challenge	problems	previous best	this run
Feynman I	51	30	33
Feynman II	33	24	25
Feynman III	15	9	10
Feynman bonus	20	0	0
Strogatz	14	3	7
total	133	66	75 (56.4%)

Table 2: Laws recovered exactly, by category. *Previous best* is the cascade run on the same development seed.

5.1 Against the published field

method	solution rate	budget
This work	56.4% (75/133)	1 seed, 360 s a problem
AlFeynman	54.1% (72/133)	10 seeds, up to 8 h a problem
GP-GOMEA	27.1%	10 seeds, up to 8 h a problem
AFP_FE	26.2%	10 seeds, up to 8 h a problem
ITEA	20.8%	10 seeds, up to 8 h a problem
AFP	20.5%	10 seeds, up to 8 h a problem
DSR	19.7%	10 seeds, up to 8 h a problem
Operon	16.0%	10 seeds, up to 8 h a problem
gplearn	15.6%	10 seeds, up to 8 h a problem

Table 3: The published ground-truth track at noise 0. The comparison is not like-for-like on seeds — see Section 10.

The benchmark allows up to eight hours for each of 133 problems, which is 1,064 hours. This run used 6.07 hours, or 0.57% of that. A ceiling of 133×360 s would be 13.3 hours; the run finished in less than half of that because most fits met the stop bar early and returned their remaining budget.

5.2 Sequencing

The run was ordered easy-first by our own signal: how fast the stop bar fired for each problem in the previous cascade run. Never the benchmark’s verdict. 42 of the first 42 problems solved in under a minute, and every law the cascade had ever solved was banked before the hard half began.

The sequencing work also exposed a harness bug: the `-problems` ordering was being applied to an already-shuffled list, so the order asked for was discarded. It affected which problems ran when, not which of them solved.

5.3 The double submission

Each fit submits two strings: `fuller`’s own final form, and a sympy-tidied version of it. The run scores 75 on the tidied string against 71 on `fuller`’s own. The tidy is worth four laws. That is the first time the pair has been anything but a diagnostic, and it says plainly that four of the 75 are reporting wins rather than search wins.

6 Why the unsolved laws failed

This analysis is of the *cascade* run, the 66-solve predecessor on the same development seed, not of the 75-solve run reported above. The cascade left 67 laws unsolved. We classified every one of them by why, not by how close it came. The exercise reframed the work more than any single mechanism did.

class	what happened	laws
A. False stop	met our stop bar with the wrong function; the search quit	3
B. No model scored	SRBench’s scorer returned <code>None</code> — our string is too long to simplify	16
C. Hit the generation cap	the cap, not the search, ended it (7 of them Strogatz)	11
D. Ran out of time	genuine search misses	53

Table 4: The cascade’s 67 unsolved laws by cause. Classes overlap: a law can be capped and unscored.

Class D dominates: 53 of 67 are the search simply not reaching the law within the time, and no amount of work on the reporting path touches them. But class B is the largest *recoverable* class, and it is entirely ours to fix — the scorer is not refusing our answer, it is failing to parse a 300-character string in the time it allows. Class C is the cheapest fix of all, and it fed directly into the generation-cap change described in Section 5: seven of those eleven are Strogatz problems that were cut off with a third of their time budget unspent.

Three diagnostics came out of the analysis, and each is a check we can run on our own output before submitting anything.

6.1 Rounding collapse

SRBench’s scorer rounds every float in the submitted expression to three decimals (and anything below 10^{-4} to zero) before comparing. A model that is the law to eleven decimals can collapse to nothing under that transform.

The worked case is Strogatz 1v2. The engine reported

$$-4096 \cdot y \cdot \tan(\tan(0.000244(x + y - 2))).$$

Since $\tan(\tan(\epsilon u)) \approx \epsilon u$ for small ϵu , and $4096 \times 0.000244 \approx 1$, this expression *is* $-y(x + y - 2)$, which is the law. It passed both halves of our stop bar. Then the scorer rounded 0.000244 to zero, the entire term collapsed, and the submitted model was the constant 0.

This is knowable before we submit, and cheaply. Round every float to three decimals exactly as `symbolic_utils.py::round_floats` does; if the result is `is_number`, the form has lost all its structure and can never match a law whatever its R^2 . A single `xreplace` over the float atoms does this in 0.2 ms, against 9.9 ms for SRBench’s per-float `subs` loop — same answer, 50 times faster, cheap enough to run inside every fit.

This is not reading the benchmark’s verdict. It is a published, deterministic transform applied to our own string, in the same class as the existing guard that asserts the reported string computes what the chromosome computes. The final-form selection now sorts on (survives rounding, size, spelling), so a surviving form beats a smaller dying one and among equals the smallest still wins.

6.2 Non-finite coverage

We evaluated the reported plain-form model of all 133 laws on the test rows.

	laws	of which solved
finite on every test row	94	66
not finite on every row	38	0

Table 5: Coverage of the reported string against solves. One law timed out of the scan.

Every single solve is on the finite side. Not one of the 38 was ever solved, in any run. In 133 laws it is a perfect predictor with no exceptions.

The cause is the protected/plain split. The engine scores the *faithful* form, where a protected logarithm returns a guard value on a negative argument. What we submit is the *plain* form, where the same expression is a raw logarithm and returns NaN on those rows. The two agree wherever the guard never fires and diverge everywhere else. Feynman II.11.28 scores $1 - R^2 = 8 \times 10^{-6}$ on the faithful form while the string we hand the benchmark is undefined on 59% of the data. Four laws are finite on 0% of rows.

This is also knowable before submitting: evaluate our own reported string on rows we already hold. We first reported 16 laws as non-finite, from a script that skipped a law on timeout and then re-read a different stage’s fit for it. Re-run with one record per law, the count is 38.

6.3 Bloat

Over all 342 fits of the cascade, a fit whose `simplified_symbolic_model` came back `None` had a median model of 323 characters. One that scored had 122. The scorer is not refusing our answer; it is failing to parse it within the time it allows. A numerically excellent model 300 characters long scores zero.

These three classes overlap rather than partition. A bloated form is also the kind that reaches a protected operator, and both are the kind that survives rounding only by accident.

7 Data-guided rewrites

The engine’s protected operators are honest about their domains and useless to a symbolic scorer. A *data-guided rewrite* is a rewrite the final form may make because the data says it holds on every row — never an identity in general, always exact on the rows the model was selected on, and always checked: the rewritten model must predict what the original predicts, to the same tolerance the rest of the rewriter is held to.

The catalogue is long and dull individually. $\text{Abs}(e)$ where e keeps one sign on every row becomes e or $-e$. A protected division whose denominator is at least 10^{-6} on every row becomes real division. A protected arcsine whose argument stays inside $[-1, 1]$ becomes the raw arcsine. $\log$ of a product with an $\exp(u)$ factor, the rest of it positive on every row, becomes $u + \log c$ — sympy will not cancel $\log(\exp u)$ without knowing u is real, and Feynman II.10.9 was the law behind $0.2 \log(\exp(5u)/5) + 0.3218\dots$ and was scored as wrong. A factor standing on both sides of a product’s line, never zero on the data, cancels; that one is what finally let Feynman II.13.23’s $\tan(\arcsin(v/c))/v$ become the law, since it is only there once the v and the c go.

Collectively the rewrites recovered ten laws the engine had already *found* and the scorer had rejected: II.11.27, III.12.43, I.18.12, I.14.4, I.30.5, III.17.37, Strogatz *glider2*, Strogatz *barmag2*, II.10.9 and I.44.4.

7.1 The worked case: III_17_37

Feynman III.17.37 was found as

$$-1.41421 \sqrt{(144 - u)^2/2} + 144, \quad u = \beta (\alpha \cos \theta + 1).$$

Since the fitted scale is $\sqrt{2}$ to nine digits, the model *is* $144 - |144 - u|$, and $u < 144$ on every row, so it is u — the law, exactly.

The existing $\sqrt{e^2} \rightarrow |e|$ rule could not see it. The radicand is a quotient, not a bare square, so the root stayed and no scorer matched the form. Splitting a positive literal out of the radicand lets the fitted scale cancel against it, and the absolute value then sheds the sign the data settles.

Measured on seed 7014, III.17.37 goes from $\log_{10} p = -30.61$ scored as nothing, to solved on both submission arms at generation 39, as $x_0 x_1 \cos(x_2) + x_0$. No run on record had ever recovered it.

7.2 The rule we declined to loosen

$\tan(u)$, $\sin(u)$, $\tanh(u)$ and $\arcsin(u)$ are all u once $|u|$ is small enough. We added the small-angle rewrite, data-guided like the rest, bounded at $|u| < 10^{-5}$ on every row. At that bound the cubic term ($u^3/3$ for $\tan$) is under 3.4×10^{-16} relative, which is inside f64’s own noise.

Strogatz *lv2* — the rounding-collapse case above — would be captured by this rule if the bound were loosened, and that is one more law. Its argument reaches 1.95×10^{-3} . At that magnitude the substitution carries a 1.3×10^{-6} relative error: ten orders of magnitude above f64 epsilon, and large enough to move the predictions.

We declined. Loosening the bound to catch *lv2* would be asserting an identity that the data says is false. The bound stays at 10^{-5} and *lv2* stays unsolved by this route.

8 Negative results

8.1 Swim lanes lost

The fourteen variation rates were one global set. We moved them onto the island record, which the device already uploads as a flat table every generation, so a per-island rule set costs rows and never a second dispatch. Each island pair then breeds under its own *lane*, written as

what it changes relative to the engine’s own schedule: a multiplier on the point-mutation and transposition rates, a multiplier on the three crossover rates, and a cleansing rate. With the cross step off, lanes never mix.

The guarantee came first: a three-lane table of the engine’s own rules produces an identical model, identical unique genes and identical individuals to no lane table at all. The engine is deterministic on a seed, so that assertion is exact.

Differentiated lanes then lost. Over ten problems on seed 7013, the uniform arm — three lanes all carrying the engine’s own schedule — solved 4; the differentiated arm, running an explorer lane and a refiner lane beside the general one, solved 3. The differentiated arm also lost Feynman I.15.10, a Lorentz law the uniform arm solved at generation 186 while the differentiated arm ran 958 generations without it.

The mechanism is sound and cheap, and the hypothesis that three rule sets running in parallel beat one was not supported. A law lost outright is a worse signal than a one-problem count difference is a good one.

8.2 The general mutation beam is exhausted

The beam takes the best individual as it stands and generates thousands of mutations of it, scored on the data, with the original never lost. The tree half of the beam is the cleanse neighbourhood enumerated whole — every function node of every used gene, promoted over each child and collapsed to a constant — plus drawn point, Dc and constant edits.

Over six near misses, a beam mutant beat its original 0.05% of the time. For comparison, changing the population size alone moved $\log_{10} p$ by up to 1.8 decades on the same problems — more than the beam ever did. The tree half is now off by default; the beat is the functional wraps alone, and the tree half survives as a knob its tests turn on.

The wrap half of the beam has its own negative result, which is instructive because it was a representational failure rather than a search failure. Wraps were originally scored on the host gene alone, to make score and graft agree by construction. That formulation cannot express four of the five laws it was aimed at: I.10.7 is $m_0/\sqrt{1-(v/c)^2}$, I.48.2 is $mc^2/\sqrt{\dots}$, I.34.1 is $\omega_0/(1-v/c)$, III.4.33 is $k_B T \cdot u/(e^u - 1)$. All four are prefactor(variables) $\times W(u)$, and with only a scalar outside the wrap the prefactor has nowhere to live. Measured over 24 fits on two seeds: 8,300 wrap candidates, 0 better, 0 grafted, 0 kept.

Moving the wrap inside the linker fixed the representation. It also exposed a different bug, because a reader should not assume score and graft agree for free: on a three-gene model, the wrap $1/(x-1)$ scored $1-R^2 = 0.0759$ and its graft computed 1.0100. A wrap that appeared to explain 92% of the variance landed as worse than the mean, because it was scored on $W(L(g_0, g_1, g_2))$ and grafted as $W(g_0)$ with the other genes set to 1.

8.3 Combination masking converted nothing

The cleanse neighbourhood is a diagonal mask: one node removed at a time. It cannot reach a nest where no single node is removable but several together are, which is the shape of the bloated near misses — Feynman I.10.7 is a 23-character law wearing 20 function calls, and dropping any single one of them loses the fit.

So we built the combination mask: draw $k \geq 2$ nodes at once, enumerate pairs while they fit the beam’s width and draw randomly past that, each step re-reading the gene so the pick is over what is left. The output is an ordinary gene, so the existing device path scores it with everything else.

Measured on one law and one fit (Feynman II.24.17), it generated 30,180 mutants a beat where the wrap-only design made 180. Eleven of them beat their original and three landed. Nothing converted into a solve.

The reason is structural and we should have seen it earlier: masking can only *remove* structure. A law that needs a term the search never built is unreachable by any amount of deletion, at any k .

8.4 Brute force does not scale the way we expected

A single timing breakdown puts 9.06 s of a 15.6 s fit in data evaluation — 58% — rather than in population handling. Larger populations therefore buy much less than the individuals-per-second figure suggests. A directly measured instance: a population of 20,000 with a 35-generation cap scored 42 of 133, against 46 for the ordinary configuration. That arm also carried a leave-one-gene-out redundancy objective, so it is not a clean population-only test; what it does show is that the combination gained four problems the smaller run lacked and lost more than it gained.

9 Benchmark integrity

The rules this run was held to, stated so they can be checked.

No restarts. One fit per problem, one search per fit. An earlier experiment that split the time budget into three independent searches scored 54 of 133 and was rejected as cheating-like — splitting the budget across independent draws is what the ten official seeds are for, and taking the best of three is not a single-seed score.

Never generate data from a known formula. The engine sees only the data SRBench provides.

The benchmark’s verdict never steers the search. Every stopping decision and every sequencing decision uses our own criterion — the $1 - R^2$ and p-value bar of Section 3. The prescore check of Section 6 is a deterministic published transform applied to our own string, not a query to the scorer.

Development seeds only. SRBench’s official ten seeds are the test set. Every number in this paper is from seeds 7001–7020.

Name-blind columns. Variables are presented as $x_0, x_1, \dots$ so no physical constant can be suggested by a variable name.

One defect in the benchmark costs us a law we cannot recover by any means. SRBench’s stored true model for Feynman II.6.15b is already rounded to $0.239 p_d \sin \theta \cos \theta / (\epsilon r^3)$. Its scorer simplifies to the double angle, which halves the constant to 0.1194, and then rounds to three decimals: the truth becomes 0.12 and our exact $3/(4\pi)$ becomes 0.119. We ran the real scorer on four forms — our fit, full precision, symbolic $3/(4\pi)$, and the double-angle form — and all four fail with the identical residual. There is no form we can submit that passes.

10 Limitations

One seed. This is one development seed, not the official ten. Same-seed arms in our own experiments differ by about ± 3 problems from the draw alone, and two seeds at identical settings have differed by 8 laws. The number would move in either direction on a full ten-seed evaluation, and the comparison in Table 3 is not like-for-like on that axis. We state 75 as a single-run development measurement, not as a rate.

We also keep it distinct from union figures. Across all configurations and seeds, the engine has solved 77 of 133 distinct laws at least once, and an analysis of laws that pass our stop bar but score as misses puts a ceiling from that line of work at 82. Neither is a single-configuration score and neither is claimed as one.

The p-value is inert here. Every $\log_{10} p$ in this run sits between -30 and -38 , against a bar of -19 , and the solved and unsolved distributions overlap completely, with medians of

−36.3 and −35.1. The p-value never binds and cannot separate a law from a near miss at this operating point; $1 - R^2$ is doing all the work of the stop bar.

The reason is the angle, not the dimension. A $\log_{10} p$ bar is dimension-free: m enters the incomplete beta of Section 3.2 as a parameter and is consumed by it, so what comes out is a probability, and $p = 10^{-19}$ means the same thing at six objectives as at nine or at nineteen thousand. That is the whole point of reporting the angle as a probability rather than as θ , and it means a bar calibrated under one objective set transfers to another unchanged. What did move is the angle itself. The bar was set on a different seed and before the precision fix that dropped the f32 floor on a true law from 1.2×10^{-10} to 1.1×10^{-14} ; four decades of headroom at the accurate end pushes every real law far past −19, which is exactly the saturation we observe. The solved and unsolved distributions then overlap because both sit deep in the tail, not because the count of objectives changed the scale.

The consequence is a real one and the remedy is not recalibration. Re-setting the bar per configuration would be treating a dimension-free quantity as though it were not, and removing objectives to move p would be deleting a measurement to improve a number. If the condition is to earn its place it has to be at an operating point where the angle discriminates — which means asking which objectives enter it, not how many. As it stands the error half of the bar, strengthened on the train side (Section 3.5), already excludes the case the p condition was introduced to catch, and on this run p adds nothing.

What we have not tested. The noisy tracks. The Feynman bonus set, where we score 0 of 20 and always have. The nine problems of the $1/\sqrt{1 - v^2/c^2}$ family, which have never been solved in any run on record — one of them, I.10.7, was found by a grafted wrap at generation 7 with test $R^2 = 1.000000$ on one seed and not on another, which is a single event and not a rate.

11 Conclusion

The result is 75 of 133 on one development seed at six minutes a fit, against a published best of 54.1% at ten seeds and up to eight hours. The measured change that moved it is a selection restriction — a tie-break in a tournament, costing one u32 per row and no additional dispatch, found by instrumenting lineage and discovering that 294,000 fresh individuals had left no descendants. What we think it is worth is broader than that, and stated as a claim rather than a result: the restriction makes space in the population for material that is not yet competitive, and a rewriter that converts back into the genotype is what fills the space with proposals rather than with noise. Section 4.5 names the experiment that would separate the two, and we have not run it.

The failure analysis is the more durable finding. Of the 67 laws unsolved before this change, 53 were the search not reaching the law at all, and no amount of work on how we write the answer touches those. The recoverable remainder is concentrated in how the answer is written and when the search is allowed to stop. Three cheap checks on our own output — does the form survive three-decimal rounding, is it finite on every row, is it short enough to parse — cover most of it, and the finiteness check held without exception across the 132 laws the scan completed.